

Rollout, Reward, Update, Repeat

How Fireworks optimizes continuous
post-training



Sinan Ozdemir

AI Education @ Fireworks
AI Lecturer, Entrepreneur, Author

My name is Sinan Ozdemir

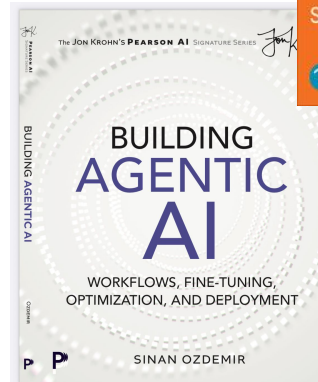
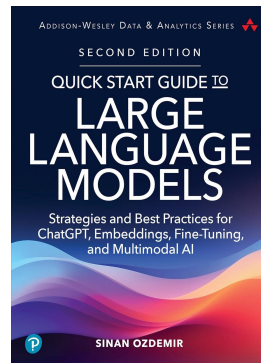
- Head of AI Education @ [Fireworks.ai](https://www.fireworks.ai)
- Former lecturer of AI - Johns Hopkins
- Founder of Kylie.ai (Acquired)
 - Early GenAI company in 2015
- Patented AI Agents with tools in 2019
- Lecturer for O'Reilly and Pearson
- Author of ML books and online series:
 - Introduction to Transformer Models
 - Quick Start Guide to LLMs
 - Building Agentic AI
- Let's connect on X / LinkedIn!



Sinan Ozdemir  
@Prof_OZ

ARTIFICIAL INTELLIGENCE COMMUNICATIONS AGENT

Inventors: CA (US); **Sinan Ozdemir**, San Francisco, CA (US)



Introduction
to Transformer
Models for NLP

Sinan Ozdemir



video

The Three Phases of Modern LLM Development

Pre-Training: “Learn the world”

- Semi/unsupervised learning - LLMs perform the next token prediction task
- Trillions of tokens worth of online and sometimes proprietary data
- Broad learning and fluency, nearly no alignment

Supervised (Mid) Training: “Mimic this pattern”

- Supervised Learning or Supervised Fine-Tuning (SFT) - learn to mimic “correct” instructions / conversations
- Goes from “just a language model” to a knowledgeable chatbot
- Partial safety, partial values

Post Training: “Optimize for behavior”

- Usually Reinforcement Learning / Reinforcement Fine-Tuning (RFT)
- Both humans and LLMs brought in to help steer an AI in the “right” direction
- Most alignment gains and most new failure risks occur here.

Pre-Training

- A necessary step in LLM development, LLMs are tasked with reading vast amounts of data with no real task other than “predict the next token”
- Trillions of tokens worth of online and sometimes proprietary data
- Broad learning and fluency, nearly no alignment is gained here other than raw knowledge
- Part of an AI lab’s secret sauce is the data they pre-train on

Common Crawl Web Extracted Text

Menu

Lemon

Introduction

The lemon, *Citrus Limon* (L.) Osbeck, is a species of small evergreen tree in the flowering plant family rutaceae. The tree's ellipsoidal yellow fruit is used for culinary and non-culinary purposes throughout the world, primarily for its juice, which has both culinary and cleaning uses. The juice of the lemon is about 5% to 6% citric acid, with a pH of around 2.2, giving it a sour taste.

Article

The origin of the lemon is unknown, though lemons are thought to have first grown in Assam (a region in northeast India), northern Burma or China. A genomic study of the lemon indicated it was a hybrid between bitter orange (sour

Please enable JavaScript to use our site.

Home
Products
Shipping
Contact
FAQ

Dried Lemons, \$3.59/pound

Organic dried lemons from our farm in California. Lemons are harvested and sun-dried for maximum flavor. Good in soups and on popcorn.

The lemon, *Citrus Limon* (L.) Osbeck, is a species of small evergreen tree in the flowering plant family rutaceae. The tree's ellipsoidal yellow fruit is used for culinary and non-culinary purposes throughout the world, pH which has both culinary. The juice of the lemon is citric acid, with a pH of a

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur in tempus quam. In mollis et ante at consectetur. Aliquam erat volutpat. Donec at lacinia est. Duis semper, magna tempor interdum suscipit, ante elit molestie urna, eget efficitur risus nunc ac elit. Fusce quis blandit lectus. Mauris at mauris a turpis tristique lacinia at nec ante. Aenean in scelerisque tellus, a efficitur ipsum. Integer justo enim, ornare vitae sem non, mollis fermentum lectus. Mauris ultrices nisl at libero porta sodales in ac orci.

function Ball(y) {
 this.radius = r;
}

Hugging Face Search models, datasets, users... Models Datasets Pricing Resources We're hiring! Log in Sign up

Dataset: bookcorpus

Dataset Structure

- Data Instances
- Data Fields
- Data Splits

Dataset Creation

- Upload to Hugging Face
- Source Data
- Annotations

Dataset Card for "bookcorpus"

Dataset Summary

Books are a rich source of both fine-grained information, how a character, an

Update on GitHub

Explore dataset

Edit Dataset Tags

Homepage: [yknzhu.wixsite.com](#) Size of downloaded dataset: 1124.87 MB

Size of dataset: 4639.00 MB Size of files used: 5753.87 MB

English Wikipedia

From Wikipedia, the free encyclopedia

[en.wikipedia.org](#) redirects here. For the Main Page, see [Main Page](#).

The **English Wikipedia** is the English-language edition of the free online encyclopedia, Wikipedia. It was founded on 15 January 2001 as Wikipedia's first edition and, as of April 2021, has the most articles of any edition, at 6,300,677.^[1] As of May 2021, 11% of articles in all Wikipedias belong to the English-language edition. This share has gradually declined from more than 50 percent in 2003, due to the growth of Wikipedias in other languages.^{[2][3]} The edition's one-billionth edit was made on 13 January 2021.^[4] The English Wikipedia has received praise for its enablement of democratization of knowledge and extent of coverage.^[5] It is the largest encyclopedia ever assembled.^[6]

The Simple English Wikipedia ([simplewiki](#)) is a variation in which most articles use only basic English vocabulary. There is also the Old English ([English/Anglo-Saxon](#)) Wikipedia ([angwiki](#)), as well as a test "incubator" version for the Middle English Wikipedia ([enmwiki](#)).

Contents [hide]

- Pioneering edition
- Size
- Wikipedians
- English Wikipedia editor numbers
- Arbitration committee

English Wikipedia

The Free Encyclopedia

Screenshot

Type of site Internet encyclopedia

Owner Wikimedia Foundation

Created by Jimmy Wales, Larry Sanger

URL [en.wikipedia.com](#)

SFT - Supervised Fine-Tuning

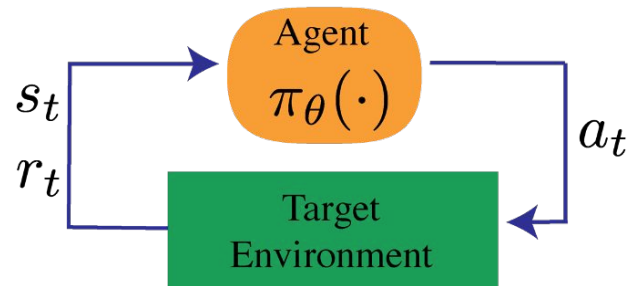
- A model learns from curated “correct” conversation examples
- This means that the conversations shown to the LLM must be correct and complete - ending with an AI message (there should be no way for the human to have the last word)
- Teaches instruction-following and stabilizes tone, style, and formatting **from the examples it's shown**

```
{  
  "messages": [  
    {"role": "system", "content": "You  
are a helpful assistant."},  
    {"role": "user", "content": "What  
is the capital of France?"},  
    {"role": "assistant", "content":  
"Paris."}  
  ]  
}
```

Fully worked out SFT conversation **includes the final assistant message**

Reinforcement Learning

- What makes Post Training unique is mainly the techniques
- **Reinforcement Learning (RL)** differs from Supervised Fine-Tuning in that **labels come after an LLM attempts a task**
- The LLM attempts to solve a task on its own (this is called a **rollout**) and then the system rewards the LLM on its work (e.g. good or bad)
- The model is then updated to try and get higher rewards



Some notation:

s_t : state

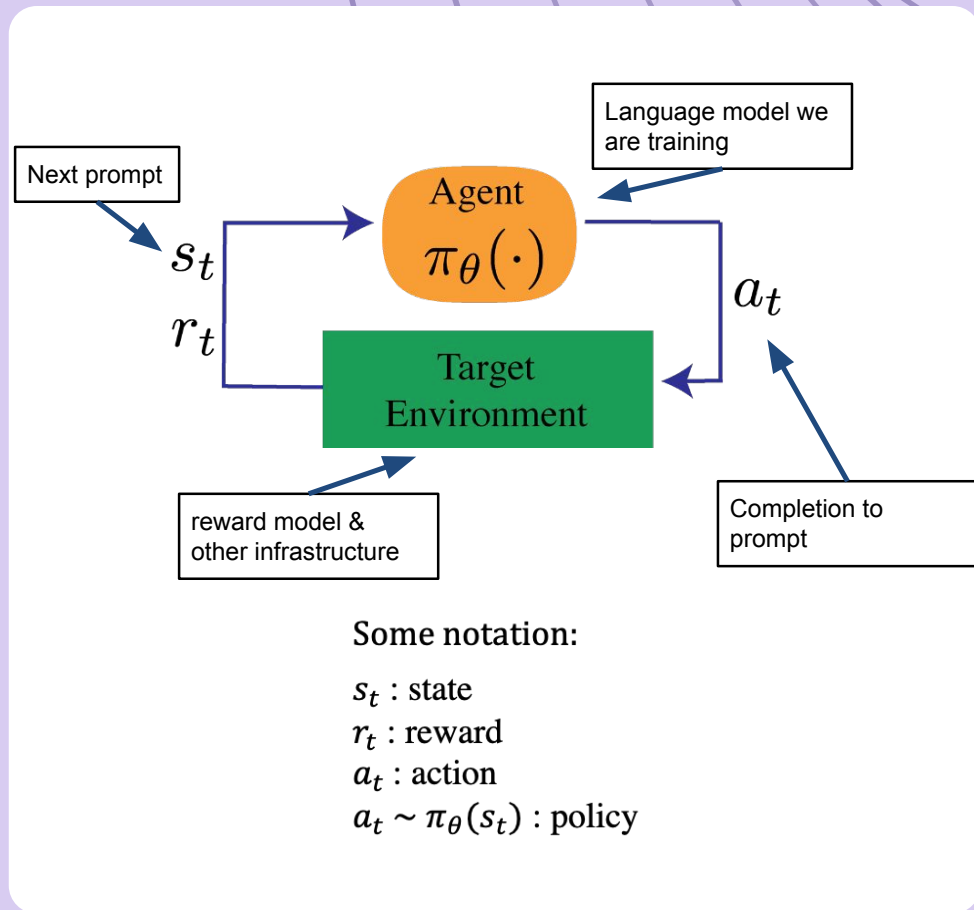
r_t : reward

a_t : action

$a_t \sim \pi_{\theta}(s_t)$: policy

The RL Loop

- RL works as a loop:
 - **Observe:** The model sees a task
 - **Act:** It generates a response
 - **Reward:** A reward model or evaluator scores the response
 - **Learn:** The model gets its weights updated
 - **Repeat:** The process happens hundreds to millions of times
- This allows for “exploration” - the model is encouraged to try new things to see how well it did



The RL Loop

ROLLOUT

Keeping the rollout
engine on-policy
with zero-KLD



REWARD

Designing dense vs.
sparse rewards



UPDATE

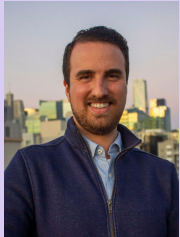
Optimized weight movement

LoRA vs full-param

REPEAT - as RL loops scale, we have to solve both modeling issues and infrastructure problems.

Rollout, Reward, Update, Repeat

Rollout



Sinan Ozdemir

AI Education @ Fireworks
AI Lecturer, Entrepreneur, Author



The RL Loop

ROLLOUT

Keeping the rollout
engine on-policy
with zero-KLD



REWARD

Designing dense vs.
sparse rewards



UPDATE

Optimized weight movement

LoRA vs full-param

REPEAT - as RL loops scale, we have to solve both modeling issues and infrastructure problems.

The rollout represents the product harness

- Each rollout is a full multi-turn episode: inference GPUs + execution environment + reward computation
- For Composer (Cursor's custom coding LLM), one rollout is an entire dev session - tool use, planning, summarizing, debugging, terminal, multi-file edits

Self-summarization inside the loop

During RL, Cursor had their agent learn to compact its own context and continue so that a 200K-context model could effectively run through **millions of tokens**, summarizing its work and restarting the window while still pursuing the task.

The many math assumption of RL

- The **trainer** nudges weights using the probability it **assumes** the model gave each token
- That must equal the probability the serving engine actually used to generate it
- When they match
 - learning signal flows accurately
- When they don't
 - you are training an off-policy model

SERVING ENGINE

probability it USED to generate token

must equal

TRAINER

probability it THINKS the model gave

gap $\neq 0$ → training off-policy (bad)

Why $\text{temp}=0$ isn't deterministic

- First of all, $\text{temp}=0$ is mathematically impossible and the fact we call it that **grinds my gears**
- You are supposed to divide logits by temperature and, historically speaking, dividing by 0 is not a mathematician's favorite thing



Why temp=0 isn't deterministic

- Floating-point addition isn't associative: $(a+b)+c \neq a+(b+c)$ at the bit level. **Change the order you sum, that's enough to flip a token**
- On an MoE model, the order changes for reasons out of your control:
 - expert matmuls pick a different kernel by token count
 - the router hits near-ties; a rounding uncertainty flips which expert fires

SERVING ENGINE

probability it USED to generate token

must equal

TRAINER

probability it THINKS the model gave

gap $\neq 0$ → off-policy

Zero-KLD = pin every load-dependent choice

PIN THE ACCUMULATION ORDER

(same operations, summed in the same sequence)

- Fixed reduction order in attention, no matter the batch
- Drop the split-K matmul trick - it reorders the sum

PIN THE CONTROL FLOW

(don't let load change which computation runs)

- One kernel for the expert matmuls, regardless of token count
- A deterministic tie-break in the router (e.g. lower index wins)

KLD: Kullback–Leibler Divergence - measure of how different two probability (token) distributions are.

Rollout, Reward, Update, Repeat

Reward



Sinan Ozdemir

AI Education @ Fireworks
AI Lecturer, Entrepreneur, Author

The RL Loop

ROLLOUT

Keeping the rollout
engine on-policy
with zero-KLD



REWARD

**Designing dense vs.
sparse rewards**



UPDATE

Optimized weight movement

LoRA vs full-param

REPEAT - as RL loops scale, we have to solve both modeling issues and infrastructure problems.

How Rewards Shape Learning

DENSE REWARD

Partial credit throughout the attempt.

- Not just a binary reward
- Easier credit assignment, (usually) faster + smoother learning
- Easier to **reward hack** - the model games by finding shortcuts to max one criteria at the cost of another

SPARSE REWARD

Feedback is binary - goal reached, or nothing.

- Tends to be easier to manufacture
- Weaker supervision: agent must explore to stumble on its way to success
- Noisier, slower but robust once it lands
- Easier to **collapse** - suddenly lose performance, memory, or ability to explore

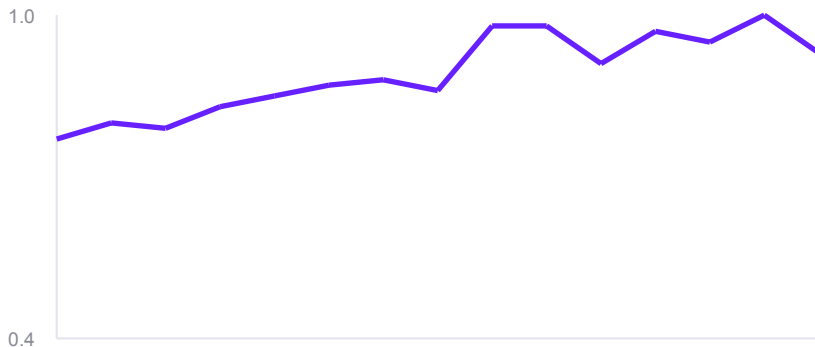
Dense examples: distance-to-target, cosine similarity, ordinal LLM-as-a-judge scores.

Sparse examples: did a test pass, did the SQL run? did the agent reach the goal, thumbs-up/down.

How Rewards Shape Learning

Countdown · DENSE

Graded on distance to target. Partial credit.



Rises fast + smooth - locks in by ~step 10

Frozen Lake · SPARSE

Reach the goal = full credit. Fail = 0.



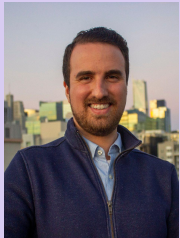
Noisier, later climb - coin-flip → reliable

Dense examples: distance-to-target, cosine similarity, ordinal LLM-as-a-judge scores.

Sparse examples: did a test pass, did the SQL run? did the agent reach the goal, thumbs-up/down.

Rollout, Reward, Update, Repeat

Update



Sinan Ozdemir

AI Education @ Fireworks
AI Lecturer, Entrepreneur, Author



The RL Loop

ROLLOUT

Keeping the rollout
engine on-policy
with zero-KLD



REWARD

Designing dense vs.
sparse rewards



UPDATE

Optimized weight movement
LoRA vs full-param

REPEAT - as RL loops scale, we have to solve both modeling issues and infrastructure problems.

Pushing new weights

- After an optimizer step, fresh weights from the trainer must reach the rollout fleet before it drifts off-policy.
 - Naive: ship the full ~1 TB (e.g.) checkpoint every step
 - RL's weight updates are sparse: most params barely move
 - Our weight compression algorithm **ARCV3** encodes the asymmetry between mantissas and exponent changes in weight deltas

Cross-region transfer per step

Full - 1024 GiB

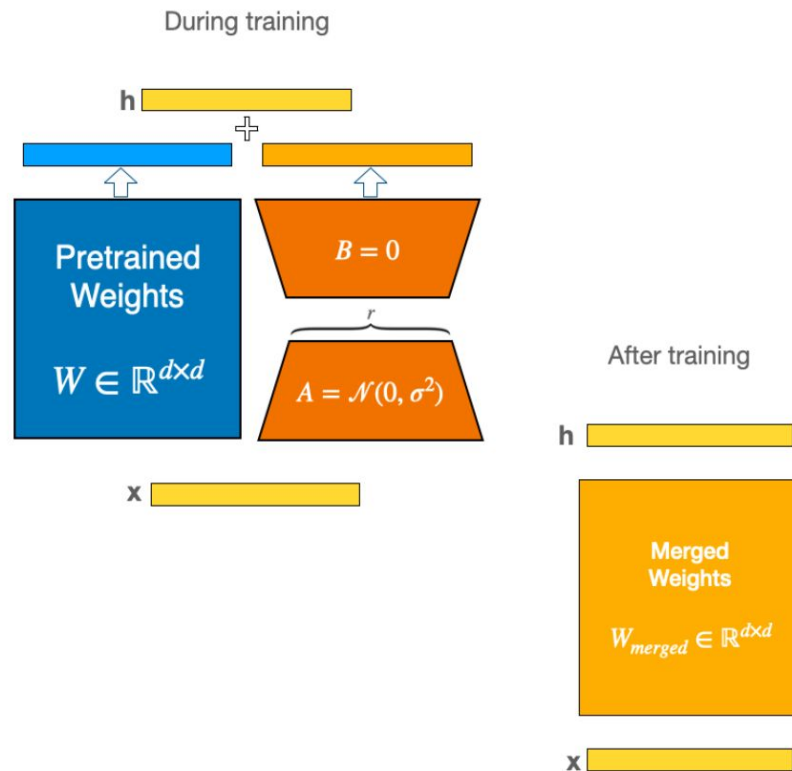
ARCV2 - 20.3 GiB (1.98%)

ARCV3 - ~2 GiB (0.19%)

~94% less transfer keeps 3-4 regions in sync with no dedicated spine.

Making Fine-tuning Efficient - LoRA

- Fireworks employs several techniques to make fine-tuning in general efficient
- One example is **LoRA** (Low-Rank Adaptation), which trains large models on smaller/single GPUs
 - You freeze the original model and train a small "adapter" - a sidecar of weights - alongside it
 - Learning this way is far cheaper than training billions of weights, and gets results close to full fine-tuning



LoRA vs Full Fine-Tuning: What the Data Says

- We have run extensive tests on training K3 and other models with LoRA. Some findings include:
 - Tweaking training recipe, data coverage, and rank can all help close the gap to full parameter fine tuning
 - The choice of reward system (e.g. dense vs sparse) in RL environments lead to different learning patterns

	Coverage	Recipe	Rank	Full Fine-Tuning (FullIFT) residual
Placement	Moved +34.0 pp	Moved +6.34 pp	Flat -0.67 pp	Unresolved +0.33 pp
Register allocation	Not tested	Not tested	Moved +8.0 pp	Unresolved +2.5 pp
Nexa VM	Moved +34.38 pp	Closed 31 pp gap	Ceiling 99.17-100%	No tested advantage
Fixed-budget mixture	Exposure sensitive	Primary +5.61 pp	Secondary +1.35 pp	Remains +4.29 pp

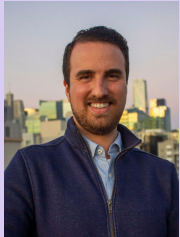
■ Primary response ■ Flat, ceiling, or no resolved gap ■ Not tested
■ Secondary response ■ Residual method gap

Source:

<https://fireworks.ai/blog/three-tests-to-run-before-you-switch-from-LoRA-to-FullIFT>

Rollout, Reward, Update, Repeat

Repeat



Sinan Ozdemir

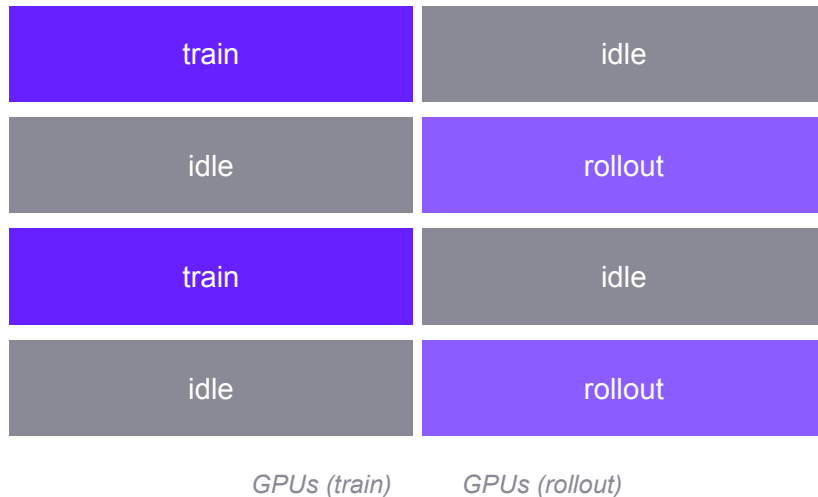
AI Education @ Fireworks
AI Lecturer, Entrepreneur, Author



Synchronous RL leaves half your GPUs idle

- The naive approach:
 - Pause training
 - Run rollouts
 - Pause rollouts
 - Train
 - Repeat
- A Composer rollout e.g. is a full agent session up to several minutes each
- While one side runs, the other is idle

SYNCHRONOUS (naive)



~50% of capacity idle, always

Async RL: decouple the workers, keep both busy

- Run training and rollout generation as independent workers that never wait on each other
- Rollouts stream in continuously; the trainer consumes finished ones and steps as soon as it has a batch
- A central reconciler manages the sample lifecycle explicitly balancing generation throughput against policy staleness

ASYNCHRONOUS



both sides busy, every step

near-full utilization

Serverless RL

ZERO-KLD numerics

Rollout engine + trainer agree bit-for-bit

DELTA / ARCv3 compression

Rollouts stay fresh across regions

ASync rollout fleet

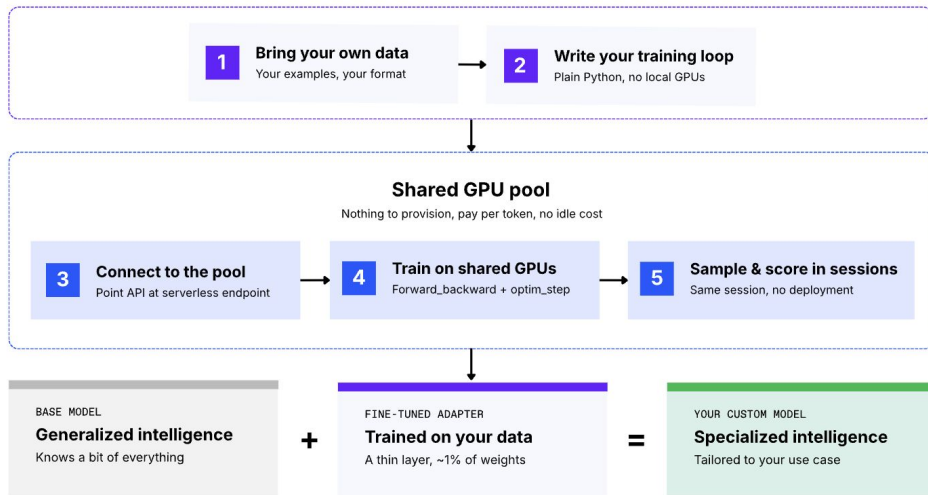
Training and rollouts stay busy

SHARED GPU pool + pay-per-token

No cluster to reserve



Fireworks Serverless Training



Source: <https://fireworks.ai/blog/K3-LoRA-Training>

Rollout, Reward, Update, Repeat

Example of Serverless Agentic RL With Kimi K3

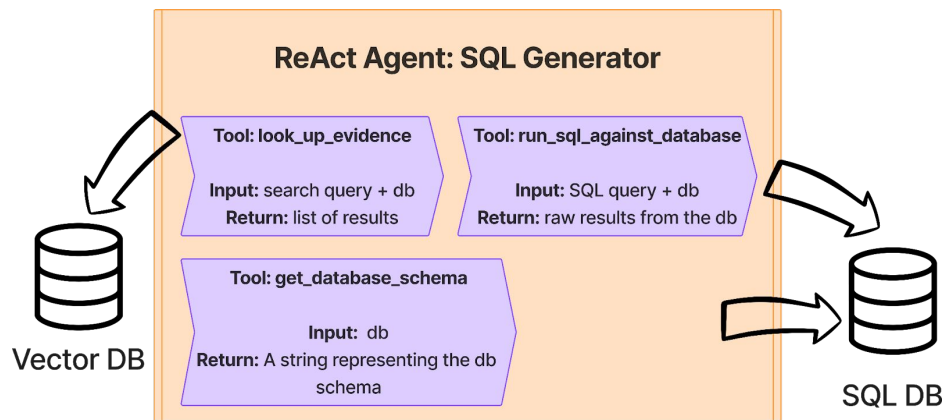


Sinan Ozdemir

AI Education @ Fireworks
AI Lecturer, Entrepreneur, Author

Our Task: Agentic Text 2 SQL

- Goal: an agent that turns natural-language questions into correct, executable SQL
- Agentic loop - inspect the schema, call tools, write SQL, self-correct on execution errors
- Realistic setting: large, messy databases where a single-shot query usually fails
- Success metric: execution accuracy - does the returned result match ground truth



Our Data: BIRD

- BIRD = Big Bench for Large-scale Database-grounded text-to-SQL
- 12,751 question/SQL pairs across 95 real databases (33.4 GB), 37 professional domains
- Emphasizes dirty database values, external knowledge, and SQL efficiency - not just schema
- Difficulty tiers: simple, moderate, challenging
- Human execution accuracy is ~93% - a hard, realistic target

Example Question:

Please list the reference names of the drivers who are eliminated in the first period in race number 20.

Example Evidence:

1. driver reference name refers to driverRef
2. first qualifying period refers to q1
3. drivers who are eliminated in the first qualifying period refers to 5 drivers with MAX(q1)
4. race number refers to raceId

Example SQL:

```
SELECT T2.driverRef FROM qualifying AS T1 INNER JOIN drivers  
AS T2 ON T2.driverId = T1.driverId WHERE T1.raceId = 20  
ORDER BY T1.q1 DESC LIMIT 5
```

Example Difficulty:

moderate

Our Model: Kimi K3

- Start with the untuned K3 agent as the baseline on a held-out BIRD slice
- K3 already knows SQL and tool-calling out of the box - strong starting point
- Record baseline execution accuracy and average turns per task before training
- This baseline is what RL has to beat - and the cost-per-task reference point



Kimi K3

accounts/fireworks/models/kimi-k3 

• Ready

Serverless

Tunable

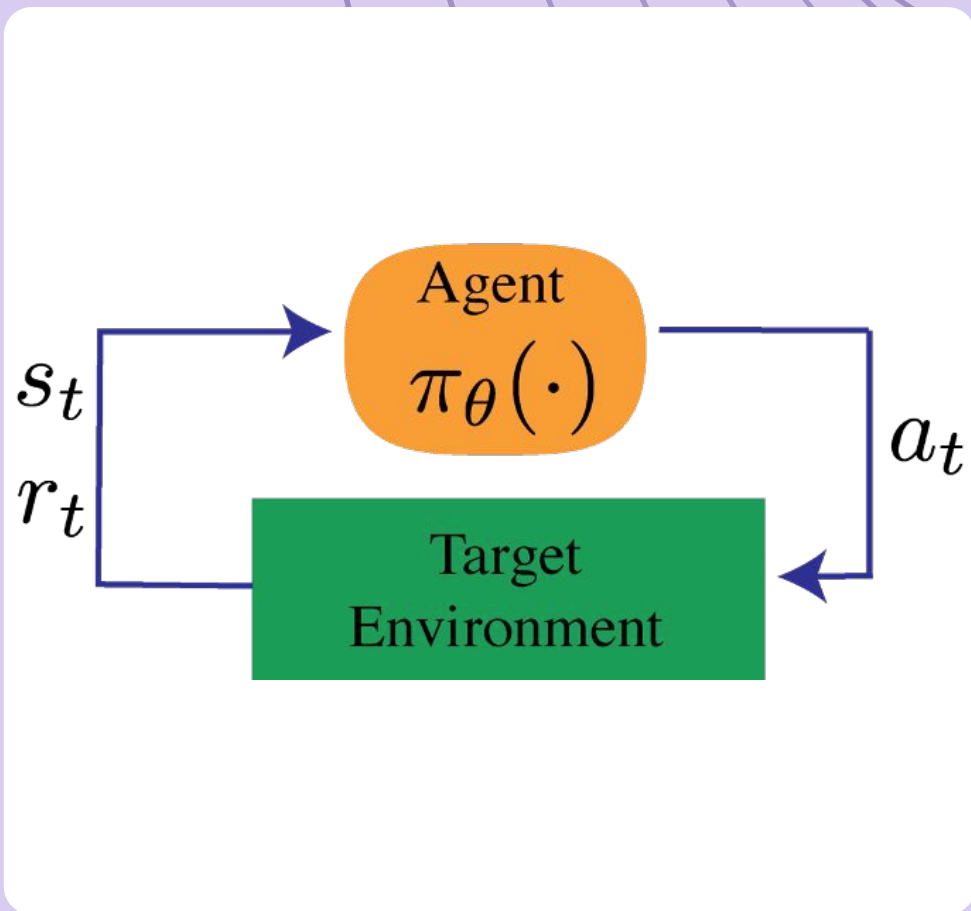
Function-calling

LLM

Vision

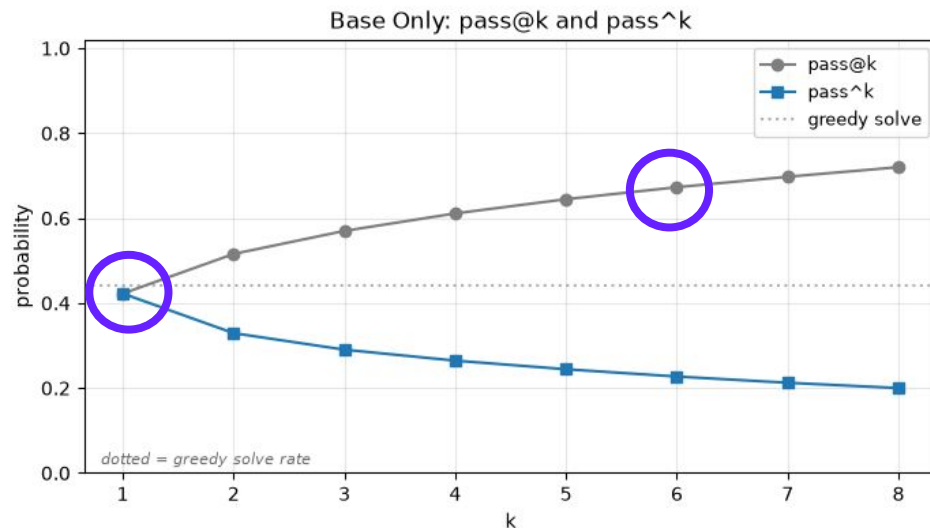
Our Technique: RL

- RL sharpens capabilities models already have rather than teaching them
 - K3 knows SQL & tool calling
- Reward is graded on results
 - 0 - SQL errors out or is wrong
 - 1 - exactly matches ground truth
 - Partial credit allowed (Jaccard)
- Execution-based: the model gets credit for a correct answer, not for matching our reference SQL



Bonus Topic: Coverage vs Consistency

- **pass@k** - $P(\geq 1 \text{ of } k \text{ runs succeeds})$
 - $1 - C(n-c, k) / C(n, k)$
 - Measures **coverage** - can the model get a right answer at all?
- **pass^k** - $P(\text{ALL of } k \text{ runs succeed})$
 - $(c / n) ^ k$
 - Measures **consistency** - will it succeed every single time?
- A gap between pass^1 and $\text{pass}@k$ (where $k \sim$ agent turns needed) indicates **headroom** to reinforce



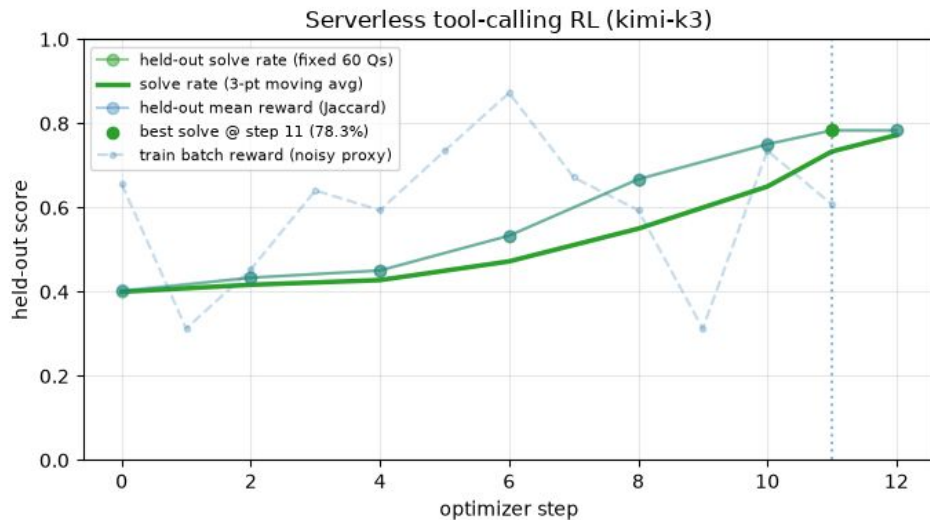
Code Time!

- We will take the same agent we saw in the first example and:
 - Set up the Fireworks Training API
 - Construct our reward system
 - Run our RL loop
 - Watch the magic (deep learning) happen



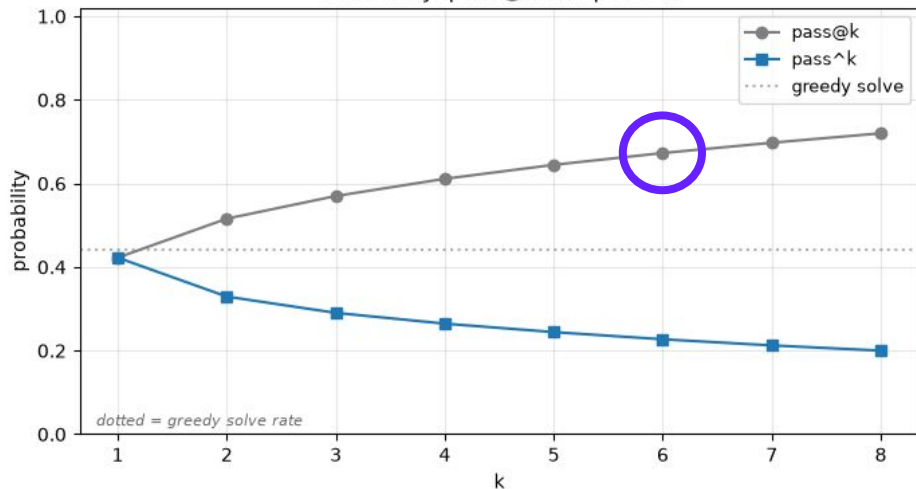
Our Results

- Solve Rate:
 - % of queries in the hold out set solved perfectly
 - 40.0% -> 78.3%
- Even though the reward was designed to give partial credit, empirically, **the reward was > 95% of the time a 0 or 1 signal**

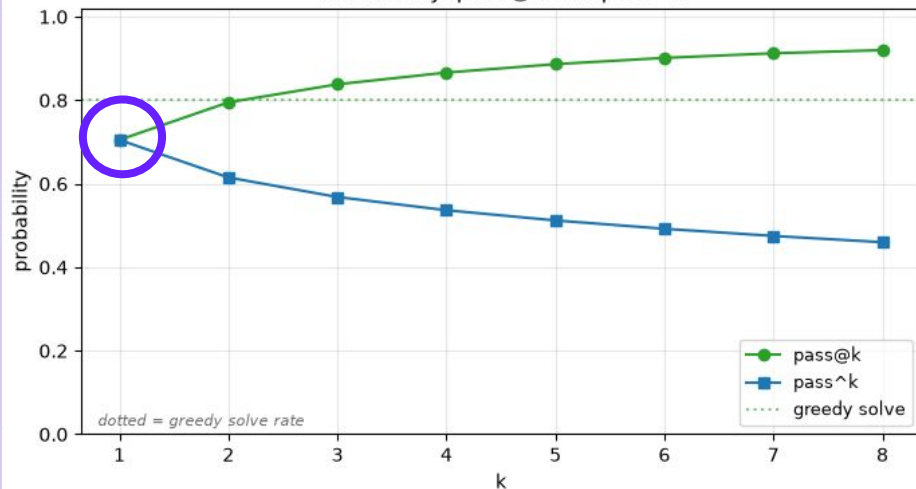


Our Results

Base Only: pass@k and pass^k



Tuned Only: pass@k and pass^k



Notice how pass¹ roughly became where pass@6 was and now we have NEW headroom (exciting!)